

Data Lab

实验报告

课程：26 fall CS Basic · Lab 1

姓名：陈域圣

学号：25800440031

日期：2026 年 10 月 9 日

目 录

摘要	2
摘要	1
一、实验目的与要求	1
1.1 实验目的	2
1.2 实验要求	2
二、实验环境与准备	2
三、实验过程：三轮测试与修复	3
3.1 环境搭建与首次构建	3
3.2 第一轮完整测试：44/110	4
3.3 第二轮测试：83/110	5
3.4 第三轮测试：110/110	5
四、各题设计思路与实现	6
五、典型问题与解决过程	38
5.1 移位量 32 的未定义行为（P5）	38
5.2 算术右移的符号扩展（P6、P9）	39
5.3 修改引入的新错误（P5、P8）	39
5.4 有符号减法的溢出（P11、P12）	39
5.5 溢出判断不充分（P13）	40
5.6 中间溢出相互抵消（P14）	40
5.7 浮点题的边界（P15、P17）	40
六、实验总结与心得	40

摘要

本实验为 CS:APP Data Lab：在大部分题目仅允许 `! ~ & ^ | + << >>` 等受限运算符、常量不得超过 `0xFF`、禁止控制流与比较运算的约束下，补全 `bits.c` 中的 19 个函数，内容覆盖二进制补码、掩码、移位、溢出与舍入，以及 IEEE 754 单精度浮点数的位级操作。

实验在 WSL2 Ubuntu-24.04 的 32 位编译环境中进行，以 `btest` 检验正确性、`check_ops.py` (dlc) 检验编码规则。开发按三轮完整测试迭代推进：第一轮 44/110，暴露出移位、符号扩展与溢出判断等 11 处错误；第二轮修复后达到 83/110；第三轮修复剩余的逻辑右移、减法溢出与浮点指数等问题，最终 110/110 全部通过，并通过全部规则检查。

本报告记录实验环境与准备过程、三轮测试的完整经过、各题设计思路与实现代码，以及典型错误的定位、原因分析与修复方案。

一、实验目的与要求

1.1 实验目的

本实验的目的是通过受限的整数运算和 IEEE 754 单精度浮点数位级操作，练习二进制补码、掩码、移位、溢出和舍入（引自实验 README）。实验的通过标准是 `./btest` 每题 Errors 为 0、总分 Total points: 110/110，且规则检查输出 All 19 functions passed operator checks.。

1.2 实验要求

只允许修改 `bits.c` 中 P1–P19 的函数体。整数题（P1–P14）仅允许 `! ~ & ^ | + << >>`，常量 0–255，禁止 `if`、`while`、比较运算符、减法、乘除取模、强制转换与 `int` 以外的类型，其中 P2 只允许 `~` 和 `&`；浮点题（P15–P17）放宽为任意整数/无符号运算与控制流，但同样禁止浮点类型与强制转换。每题还有最大操作数上限，由 `check_ops.py` 逐题检查。各题概况见表 1。

表 1 Data Lab 题目总览

题号	函数	分值	核心方法
P1	<code>signMask</code>	1	1 左移 31 位取最高位掩码
P2	<code>bitXor</code>	2	德摩根律改写异或

题号	函数	分值	核心方法
P3	negativePart	3	符号掩码 × 补码绝对值
P4	copyByteWithin	4	移位取字节 + 掩码清零回填
P5	logicalShift	4	算术右移 + 掩码清理符号位
P6	swapNibblePairs	4	半字节掩码交换并清理符号扩展
P7	secondLowestZeroBit	4	$\sim x \& (x+1)$ 取最低 0 位技巧用两次
P8	oddParity	5	异或折叠取奇偶
P9	rotateRightBits	5	逻辑右移清理 + 左移 $(32-n) \& 31$
P10	roundEvenPow2	5	偏置舍入后清零低 n 位
P11	midpointTowardFirst	5	无溢出平均数 + 同号异号分开比较
P12	isBetweenEitherOrder	7	无溢出符号比较
P13	mul5Sat	7	绝对值阈值法判溢出
P14	classifyAdd3	7	进位 + 符号位高位计数法
P15	floatScaleThreeHalves	7	尾数乘 3 归一化 + 向偶舍入
P16	floatRoundEven	10	定点右移取整 + 再归一化
P17	float_i2f	10	最高位定位 + 丢弃位舍入
P18	bitCount	10	分治计数（四层掩码）
P19	bitReverse	10	分治反转（五层对换）

二、实验环境与准备

实验在支持 32 位编译的 x86-64 Linux 环境中完成，工具链见表 2。

表 2 实验环境与工具

项目	内容
操作系统	Windows 11 + WSL2 (Ubuntu-24.04)
编译方式	gcc -O -Wall -fwrapv -m32 (32 位编译)
构建	GNU Make: make all 生成 btest、ishow、fshow
正确性测试	./btest (支持 -f 单题、-1/-2/-3 固定参数)
规则检查	./check_ops.py bits.c (内部调用 dlc)
位表示查看	./ishow (整数)、./fshow (浮点)
编辑器	VSCoDe (Remote-WSL)

项目	内容
版本管理	git, 推送 GitHub 仓库自动评测

准备过程：按 README 安装 32 位编译支持（gcc-multilib、libc6-dev-i386）；运行脚本时遇到 Permission denied，按提示执行 `chmod +x check_ops.py dlc` 补上执行权限，并把 .vscode 配置一并入库方便远程开发；随后 `make clean && make all` 一次通过（图 1）。README 特别提醒 `btest` 不会在源文件修改后自动重编译，因此后续每次修改 `bits.c` 后都重新构建再测试。

三、实验过程：三轮测试与修复

3.1 环境搭建与首次构建

图 1 为首次构建的输出，三条 `gcc` 命令分别以 `-O -Wall -fwrapv -m32` 编译出 `btest`、`fshow` 与 `ishow`。此时 `bits.c` 尚未填写，首次 `./btest` 应输出 `Total points: 0/110`，作为后续测试的基线。

```
cysjake@CYS-HELPER:~/26fall-cs basic/lab1/Datalab-cys$ make clean
rm -f *.o btest fshow ishow *~
cysjake@CYS-HELPER:~/26fall-cs basic/lab1/Datalab-cys$ make all
gcc -O -Wall -fwrapv -m32 -o btest bits.c btest.c decl.c tests.c -lm
gcc -O -Wall -fwrapv -m32 -o fshow fshow.c
gcc -O -Wall -fwrapv -m32 -o ishow ishow.c
```

图 1 `make clean && make all` 编译通过

3.2 第一轮完整测试：44/110

完成 19 题初版后进行第一次完整测试（图 2）。`signMask`、`bitXor`、`negativePart`、`copyByteWithin`、`secondLowestZeroBit`、`floatRoundEven`、`bitCount`、`bitReverse` 共 8 题一次通过，得 44 分；其余 11 题报错。失败样例几乎全部集中在 `INT_MIN`（`0x80000000`）这类边界输入上，例如：

```
ERROR: Test logicalShift(-2147483648[0x80000000],0[0x0]) failed...
...Gives 0[0x0]. Should be -2147483648[0x80000000]
ERROR: Test swapNibblePairs(-2147483648[0x80000000]) failed...
...Gives 0[0x0]. Should be 134217728[0x80000000]
ERROR: Test floatScaleThreeHalves(8388608[0x80000000]) failed...
...Gives 20971520[0x14000000]. Should be 12582912[0xc0000000]
```

这些错误的共性是：算术右移的符号扩展、移位量为 32 的未定义行为、有符号减法溢出，以及浮点阶码处理不当，逐题分析见第五章。

```

cysjake@CYS-HELPER:~/26fall-cs basic/lab1/Datalab-cys$ ./btest
Score  Rating  Errors  Function
1      1      0      signMask
2      2      0      bitXor
3      3      0      negativePart
4      4      0      copyByteWithin
ERROR: Test logicalShift(-2147483648[0x80000000],0[0x0]) failed...
...Gives 0[0x0]. Should be -2147483648[0x80000000]
ERROR: Test swapNibblePairs(-2147483648[0x80000000]) failed...
...Gives 0[0x0]. Should be 134217728[0x80000000]
4      4      0      secondLowestZeroBit
ERROR: Test oddParity(-2147483648[0x80000000]) failed...
...Gives 1431655765[0x55555555]. Should be 0[0x0]
ERROR: Test rotateRightBits(-2147483648[0x80000000],2147483647[0x7fffffff
f]) failed...
...Gives -1[0xffffffff]. Should be 1[0x1]
ERROR: Test roundEvenPow2(1073741823[0x3fffffff],1[0x1]) failed...
...Gives 1073741822[0x3ffffffe]. Should be 1073741824[0x40000000]
ERROR: Test midpointTowardFirst(-2147483648[0x80000000],-2147483647[0x80
000001]) failed...
...Gives -2147483647[0x80000001]. Should be -2147483648[0x80000000]
ERROR: Test isBetweenEitherOrder(-2147483648[0x80000000],-2147483648[0x8
0000000],0[0x0]) failed...
...Gives 0[0x0]. Should be 1[0x1]
ERROR: Test mul5Sat(2147483647[0x7fffffff]) failed...
...Gives 2147483643[0x7ffffffb]. Should be 2147483647[0x7fffffff]
ERROR: Test classifyAdd3(-2147483648[0x80000000],-2147483647[0x80000001]
,2147483647[0x7fffffff]) failed...
...Gives -1[0xffffffff]. Should be 0[0x0]
ERROR: Test floatScaleThreeHalves(8388608[0x800000]) failed...
...Gives 20971520[0x1400000]. Should be 12582912[0xc00000]
10     10     0      floatRoundEven
ERROR: Test float_i2f(-2147483648[0x80000000]) failed...
...Gives -813694976[0xcf800000]. Should be -822083584[0xcf000000]
10     10     0      bitCount
10     10     0      bitReverse
Total points: 44/110
cysjake@CYS-HELPER:~/26fall-cs basic/lab1/Datalab-cys$ █

```

图 2 第一轮完整测试：44/110

3.3 第二轮测试：83/110

针对第一轮的错误逐题修复：oddParity 补上取末位并取反；rotateRightBits 增加符号位清理；roundEvenPow2 改为偏置舍入；isBetweenEitherOrder 重写为无溢出的符号比较；floatScaleThreeHalves 修正阶码处理；float_i2f 补上尾数取高 23 位。第二轮测试 83/110（图 3），仍剩 5 题错误：logicalShift、swapNibblePairs、midpointTowardFirst、mul5Sat、classifyAdd3。

```

cysjake@CYS-HELPER:~/26fall-cs basic/lab1/Datalab-cys$ ./btest
Score  Rating  Errors  Function
1      1      0      signMask
2      2      0      bitXor
3      3      0      negativePart
4      4      0      copyByteWithin
ERROR: Test logicalShift(-2147483648[0x80000000],1[0x1]) failed...
...Gives 0[0x0]. Should be 1073741824[0x40000000]
ERROR: Test swapNibblePairs(-2147483648[0x80000000]) failed...
...Gives -134217728[0xf8000000]. Should be 134217728[0x80000000]
4      4      0      secondLowestZeroBit
5      5      0      oddParity
5      5      0      rotateRightBits
5      5      0      roundEvenPow2
ERROR: Test midpointTowardFirst(-2147483648[0x80000000],2147483647[0x7fffff]) failed...
...Gives 0[0x0]. Should be -1[0xffffffff]
7      7      0      isBetweenEitherOrder
ERROR: Test mul5Sat(2147483647[0x7fffff]) failed...
...Gives 2147483643[0x7fffffb]. Should be 2147483647[0x7fffff]
ERROR: Test classifyAdd3(-2147483648[0x80000000],-2147483648[0x80000000],-2147483648[0x80000000]) failed...
...Gives 0[0x0]. Should be -1[0xffffffff]
7      7      0      floatScaleThreeHalves
10     10     0      floatRoundEven
10     10     0      float_i2f
10     10     0      bitCount
10     10     0      bitReverse
Total points: 83/110

```

图 3 第二轮测试：83/110

3.4 第三轮测试：110/110

继续调整剩余 5 题：logicalShift 补回漏掉的右移；swapNibblePairs 清理右移补进来的 1；midpointTowardFirst 改为同号、异号分开比较；mul5Sat 改用阈值法判断溢出并修正负阈值差一；classifyAdd3 重写为高位计数法。第三次 ./btest 全部 19 行 Errors 均为 0，Total points: 110/110（图 4）；随后 ./check_ops.py bits.c 逐题核对操作数上限，输出 All 19 functions passed operator checks.，实验完成。三轮对比见表 3。

```

cysjake@CYS-HELPER:~/26fall-cs basic/lab1/Datalab-cys$ ./btest
Score  Rating  Errors  Function
1      1      0      signMask
2      2      0      bitXor
3      3      0      negativePart
4      4      0      copyByteWithin
4      4      0      logicalShift
4      4      0      swapNibblePairs
4      4      0      secondLowestZeroBit
5      5      0      oddParity
5      5      0      rotateRightBits
5      5      0      roundEvenPow2
5      5      0      midpointTowardFirst
7      7      0      isBetweenEitherOrder
7      7      0      mul5Sat
7      7      0      classifyAdd3
7      7      0      floatScaleThreeHalves
10     10     0      floatRoundEven
10     10     0      float_i2f
10     10     0      bitCount
10     10     0      bitReverse
Total points: 110/110
cysjake@CYS-HELPER:~/26fall-cs basic/lab1/Datalab-cys$ █

```

图 4 第三轮测试：110/110 全部通过

四、各题设计思路与实现

以下按题号顺序记录各题的设计思路与最终实现，说明文字尽量保留 bits.c 中的注释原话（引号内均为代码注释原文）。

P1 signMask (1 分)

```

int signMask(void) {
    return 1<<31;    //1 左移 31 位, 后面位置补 0;
}

```

P2 bitXor (2 分)

```

int bitXor(int x, int y) {
    return ~(~x & ~y) & ~(x & y);
}

```

P3 negativePart (3 分)

```

int negativePart(int x){

```

```
int mask=x>>31;
return mask&(~x+1);
}
```

P4 copyByteWithin (4分)

）后回填。

```
int copyByteWithin(int x, int src, int dst) {
    int src_shift=src<<3;
    int dst_shift=dst<<3;
    int src_byte=(x>>src_shift)&0xFF;
    int mask=~(0xFF<<dst_shift);
    return (x&mask)|(src_byte<<dst_shift);
}
```

P5 logicalShift (4分)

算术右移后再用掩码清掉补进来的符号位。掩码写成 $\sim(((1<<31)>>n)<<1)$ 。这道题前后改了三版，踩坑记录见 5.1 与 5.3 节。

```
int logicalShift(int x, int n) {
    int mask=~(((1 << 31) >> n) << 1); //避免错误移位 32 (n=0)
    return (x>>n)& mask; //第二次尝试时漏掉了右移操作!!!
}
```

P6 swapNibblePairs (4分)

```
int swapNibblePairs(int x) {
    int mask_low = 0x0F|(0x0F << 8)|(0x0F << 16)|(0x0F << 24);
    int mask_high = ~mask_low;
    return ((x & mask_low) << 4) | (((x & mask_high) >> 4)&mask_low);
}
```

P7 secondLowestZeroBit (4分)

```
int secondLowestZeroBit(int x) {
    int x1=(~x&(x+1));
    int x2=(x1|x);
    int x3=(~x2&(x2+1));
    return x3;
}
```

P8 oddParity (5分)

最后 $(x\&1)^1$ 把它变成题目要求的定义（偶数个 1 返回 1）。

```
int oddParity(int x) {
    x^=x>>16;
    x^=x>>8;
```

```

x^=x>>4;
x^=x>>2;
x^=x>>1;
return (x&1)^1;
}

```

P9 rotateRightBits (5 分)

```

int rotateRightBits(int x, int n) {
    n=n&31;
    int logical_shifted=(x >> n) & ~(((1<<31)>>n) << 1);
    int left_shifted=x << ((32 + ~n + 1) & 31);
    return logical_shifted | left_shifted;
}

```

P10 roundEvenPow2 (5 分)

```

int roundEvenPow2(int x, int n) {
    int half_minus_1=(1<<(n+~0))+~0; //2^(n-1)
    int q=(x>>n)&1;
    int mask = (1 << n) + ~0;
    return (x +half_minus_1+q) & ~mask;
}

```

P11 midpointTowardFirst (5 分)

```

int midpointTowardFirst(int x, int y) {
    int avg=(x & y) + ((x ^ y)>>1);
    int odd=(x^y) & 1;
    int sign_x = x >>31;
    int sign_y = y >>31;
    int diff_sign=sign_x^sign_y;
    int diff=x+~y+1;
    int same_gt = ~(diff >> 31) & 1;
    int diff_gt = (~sign_x) & sign_y;
    int x_gt_y =(diff_sign&diff_gt)|(~diff_sign&same_gt);
    return avg +(odd & x_gt_y);
}

```

P12 isBetweenEitherOrder (7 分)

。每个比较都拆成两种情况：两数同号时看差值的符号位，异号时符号本身就决定大小；其中异或结果为 0 意味着同号，差值的符号位异或溢出标志后取反即得布尔值。

```

int isBetweenEitherOrder(int x, int a, int b) {
    int dxa = x + ~a + 1;
    int of_a = (x ^ a) & (x ^ dxa);
}

```

```

int x_ge_a = ~((dxa ^ of_a) >> 31);
int dxb = x + ~b + 1;
int of_b = (x ^ b) & (x ^ dxb);
int x_ge_b = ~((dxb ^ of_b) >> 31);
int dax = a + ~x + 1;
int of_ax = (a ^ x) & (a ^ dax);
int a_ge_x = ~((dax ^ of_ax) >> 31);
int dbx = b + ~x + 1;
int of_bx = (b ^ x) & (b ^ dbx);
int b_ge_x = ~((dbx ^ of_bx) >> 31);
return ((x_ge_a & b_ge_x) | (x_ge_b & a_ge_x)) & 1;
}

```

P13 mul5Sat (7分)

最终按 $|x|$ 是否超过 $\text{INT_MAX}/5$ ($0x19999999$) 判断, 阈值用 $0x19$ 、 $0x99$ 移位拼出; 绝对值用 $(x+\text{sign})^{\text{sign}}$ 一步得到; 饱和值用 $\sim(\text{sign}^{(1\ll 31)})$ 同时覆盖两个方向。

```

int mul5Sat(int x) {
    int sign = x >> 31;
    int abs_x = (x + sign) ^ sign;
    int pos_thresh = (0x19 << 24) | (0x99 << 16) | (0x99 << 8) | 0x99;
    int diff = abs_x + ~pos_thresh + 1;
    int diff_sign = diff >> 31;
    int overflow = ~diff_sign & !!diff;
    int mask = ~overflow + 1;
    int mul5 = (x << 2) + x;
    int sat_val = ~(sign ^ (1 << 31));
    return (mask & sat_val) | (~mask & mul5);
}

```

P14 classifyAdd3 (7分)

思路: 每次加法用 $((a\&b)|((a|b)\&\sim s))\gg 31\&1$ 提取进位, 把三个符号位与进位组成高部; 真实和 = 高部 $\times 2^{32}$ + 低 32 位, 高部不在 $\{0, -1\}$ 中必溢出; 高部为 0 而低部为负时是正溢出, 高部为 -1 而低部非负时是负溢出。

```

int classifyAdd3(int x, int y, int z) {
    int s1 = x + y;
    int c1 = ((x & y) | ((x | y) & ~s1)) >> 31 & 1;
    int s2 = s1 + z;
    int c2 = ((s1 & z) | ((s1 | z) & ~s2)) >> 31 & 1;
    int carry = c1 + c2;
    int sx = (x >> 31) & 1;
    int sy = (y >> 31) & 1;
    int sz = (z >> 31) & 1;
    int sign_sum = sx + sy + sz;
    int high = carry + ~sign_sum + 1;
    int sum_low = s2;
    int sign_low = sum_low >> 31;
}

```

```

int low_neg = sign_low & 1;
int high_sign = high >> 31;
int high_nonzero = !!high;
int high_gt_zero = high_nonzero & ~high_sign & 1;
int high_eq_zero = !high;
int overflow_pos = high_gt_zero | (high_eq_zero & low_neg);
int high_plus_1 = high + 1;
int high_plus_1_sign = high_plus_1 >> 31;
int high_lt_minus_1 = high_plus_1_sign & 1;
int high_eq_minus_1 = !(high + 1);
int low_nonneg = ~sign_low & 1;
int overflow_neg = high_lt_minus_1 | (high_eq_minus_1 & low_nonneg);
int neg_result = overflow_neg << 31 >> 31;
return overflow_pos | neg_result;
}

```

P15 floatScaleThreeHalves (7分)

```

unsigned floatScaleThreeHalves(unsigned uf) {
    unsigned sign=uf&0x80000000;
    unsigned exp=(uf>>23)&0xFF;
    unsigned frac=uf&0x7FFFFFFF;
    unsigned m,t,E,half,tail;
    if(exp==0xFF){ return uf; }
    if(exp==0){
        t = frac * 3;
        tail = t >> 1;
        if (t & 1) { tail += tail & 1; }
        if (tail & 0x800000) { return sign | (1 << 23) | (tail & 0x7FFFFFFF); }
        return sign | tail;
    }
    m = 0x800000 | frac;
    t = m * 3;
    if (t >= 0x2000000) {
        E = exp + 1;
        tail = t >> 2;
        if (t & 2) {
            unsigned rem = t & 3;
            if (rem > 2 || (rem == 2 && (tail & 1))) { tail++; }
        }
    } else {
        E = exp;
        tail = t >> 1;
        if (t & 1) { if (tail & 1) { tail++; } }
    }
    if (tail & 0x1000000) { tail >>= 1; E++; }
    if (E >= 0xFF) { return sign | 0x7F800000; }
    return sign | (E << 23) | (tail & 0x7FFFFFFF);
}

```

P16 floatRoundEven (10 分)

) 。

```
unsigned floatRoundEven(unsigned uf) {
    unsigned sign=uf&0x80000000;
    unsigned exp=(uf>>23)&0xFF;
    unsigned frac=uf&0x7FFFFFFF;
    unsigned m,sh,v,rem,half,p;
    if(exp==0xFF){ return uf; }
    if(exp<126){ return sign; }
    if(exp>=150){ return uf; }
    m=frac|0x800000;
    sh=150-exp;
    v=m>>sh;
    rem=m&((1<<sh)-1);
    half=1<<(sh-1);
    if((rem>half)||((rem==half)&&(v&1))){ v=v+1; }
    if(!v){ return sign; }
    p=23;
    while(!(v&(1<<p))){ p--; }
    return sign|((p+127)<<23)|((v<<(23-p))&0x7FFFFFFF);
}
```

P17 float_i2f (10 分)

； pos≤23 时左移对齐、精确表示；否则右移丢弃低位，尾数 &0x7FFFFFFF。

```
unsigned float_i2f(int x) {
    unsigned sign=0;
    unsigned abs;
    unsigned frac,rem,half;
    int pos,sh,exp;
    if(!x){ return 0; }
    if(x<0){ sign=0x80000000; abs=~x+1; } else { abs=x; }
    pos=31;
    while(!(abs&(1<<pos))){ pos--; }
    exp=pos+127;
    if(pos<=23){
        frac=(abs<<(23-pos))&0x7FFFFFFF;
    }else{
        sh=pos-23;
        frac=(abs>>sh)&0x7FFFFFFF;
        rem=abs&((1<<sh)-1);
        half=1<<(sh-1);
        if((rem>half)||((rem==half)&&(frac&1))){ frac++; }
        if(frac==0x800000){ frac=0; exp++; }
    }
    return sign|(exp<<23)|frac;
}
```

P18 bitCount (10 分)

```

int bitCount(int x) {
    int m1 = 0x55; m1 = m1 | (m1 << 8); m1 = m1 | (m1 << 16);
    int m2 = 0x33; m2 = m2 | (m2 << 8); m2 = m2 | (m2 << 16);
    int m4 = 0x0F; m4 = m4 | (m4 << 8); m4 = m4 | (m4 << 16);
    int m8 = 0xFF; m8 = m8 | (m8 << 16);
    int m16 = 0xFF; m16 = m16 | (m16 << 8);
    x = (x & m1) + ((x >> 1) & m1);
    x = (x & m2) + ((x >> 2) & m2);
    x = (x & m4) + ((x >> 4) & m4);
    x = (x & m8) + ((x >> 8) & m8);
    return (x & m16) + ((x >> 16) & m16);
}

```

P19 bitReverse (10 分)

分治反转：依次交换距离为 1、2、4、8、16 的位。五个掩码由 0x00FF00FF 出发用异或逐级推出（ $m3=m4 \oplus (m4 \ll 4)$ 得 0x0F0F0F0F， $m2=m3 \oplus (m3 \ll 2)$ 得 0x33333333， $m1=m2 \oplus (m2 \ll 1)$ 得 0x55555555），掩码构造共用 10 个操作，恰好满足 34 的上限；最后一层右移后与 0x0000FFFF 相与，清理算术右移补进来的符号位。

```

int bitReverse(int x) {
    int m1, m2, m3, m4, m5;
    m4 = 0xFF | (0xFF << 16);
    m3 = m4 ^ (m4 << 4);
    m2 = m3 ^ (m3 << 2);
    m1 = m2 ^ (m2 << 1);
    m5 = 0xFF | (0xFF << 8);
    x = ((x >> 1) & m1) | ((x & m1) << 1);
    x = ((x >> 2) & m2) | ((x & m2) << 2);
    x = ((x >> 4) & m3) | ((x & m3) << 4);
    x = ((x >> 8) & m4) | ((x & m4) << 8);
    x = ((x >> 16) & m5) | (x << 16);
    return x;
}

```

五、典型问题与解决过程

5.1 移位量 32 的未定义行为 (P5)

现象：第一轮 `logicalShift(-2147483648, 0)` 返回 0，应为 0x80000000。原因：初版掩码写成 $0xFFFFFFFF \ll (32-n)$ ， $n=0$ 时移位量为 32。C 语言规定移位量必须小于位宽，实际机器上按对 32 取模处理，得到的是错误掩码，把结果全部清零。修复后掩码改为 $\sim(((1 \ll 31) \gg n) \ll 1)$ 。验证： $n=0$ 时 $(1 \ll 31) \ll 1$ 溢出丢弃得 0，掩码为全 1； $n=31$ 时掩码为 1，恰好只保留最低位。

5.2 算术右移的符号扩展 (P6、P9)

现象：swapNibblePairs(-2147483648) 返回 0xF8000000（应为 0x80000000）；rotateRightBits(-2147483648, 2147483647)修复套路：凡是右移后只保留低位的场合，右移后再与低位掩码相与，把补进来的 1 清掉。这一招在 P5、P6、P9、P19 中反复使用。

5.3 修改引入的新错误（P5、P8）

第二轮 logicalShift(-2147483648, 1)教训：每修一处都要重新完整测试，修复本身也可能引入新错误。

5.4 有符号减法的溢出（P11、P12）

地二轮测试 midpointTowardFirst(-2147483648, 2147483647)。

isBetweenEitherOrder 同样败在减法上，第一轮 isBetweenEitherOrder(-2147483648, -2147483648, 0)。最终每个比较都拆成同号（看差值符号）与异号（符号位直接决定）两种情况，并用异或结果是否为 0 判断同号。

5.5 溢出判断不充分（P13）

第十三题 mul5Sat 修复：|x| 超过 INT_MAX/5 即溢出，阈值 0x19999999 用小常量移位拼出；负方向阈值一度写成 0xE6666666，但这样恰好差一，导致 x=-429496730 漏判，改为 0xE6666667 后通过。但云端评测又出问题，仔细一看没认真审题！最后使用小数移位方式实现乘法得到大数。

5.6 中间溢出相互抵消（P14）

现象：第一轮 classifyAdd3(-2147483648, -2147483647, 2147483647)数值溢出相互抵消导致判断出问题，后面改进了判断。

5.7 浮点题的边界（P15、P17）

floatScaleThreeHalves 第一轮在最小规格化数上出错：floatScaleThreeHalves(0x800000)尾数乘 3 后位数增加只是归一化，不代表数值变大，阶码不能加。重写后按 t=3M 的位数分两支处理。

float_i2f 第一轮在 INT_MIN 上出错：float_i2f(-2147483648) 返回 0xCF800000（应为 0xCF000000）。原因是右移后的尾数没有清掉高位，修复为 (abs>>sh)&0x7FFFFFFF，舍入进位判断相应从 0x1000000 改为 0x800000。

六、实验总结与心得

符号位是大多数错误的根源。第一轮 11 个错误几乎全部由 0x80000000 触发：算术右移补 1（P6、P9）、移位 32 位（P5）、异号相减溢出（P11、P12）。凡是右移后只用低位的场合，必须先用掩码把补进来的 1 清理掉，这成了贯穿多题的固定套路。

溢出的检查需要仔细！！乘法（P13）、连加（P14）、减法（P11、P12）的溢出表现各不相同：截断后符号可能不变、中间溢出可能相互抵消、最小减最大直接爆掉。可靠的做法要么限制输入范围（阈值法），要么把进位与符号位显式记账（高位计数法），要么把运算拆到不会溢出的域（无溢出平均数公式 $(x \& y) + ((x \wedge y) \gg 1)$ ）。

舍入统一成一套模式。右移丢弃低位后，比较余数与半值：余数更大，或恰好一半且保留部分为奇时进一。P10、P16、P17 三次使用同一套判断，写熟后出错率大大降低。

关于 AI 辅助的说明。P12、P13、P14 三题在多轮独立尝试失败后，借助 AI 完成了无溢出符号比较、阈值法与高位计数法，并已在代码注释中如实标注；其余各题均为独立完成。实验报告代码部分的格式美观由 AI 辅助生成。

通过本实验，我对补码表示、移位运算的未定义行为、溢出的本质，以及 IEEE 754 的编码与向偶舍入有了远超课堂公式的具体理解；同时我也体会到，边界测试（尤其是 INT_MIN）应当从写下第一行代码时就纳入考虑，而不是等测试报错后再补，以及我要认真审题！！